



AL-2001 Artificial Intelligence Lab # 6

Objectives:

- Classes, Objects, Constructor

Note: Carefully read the following instructions (*Each instruction contains a weightage*)

1. First think about statement problems and then write your logic on Copy / Notebook.
2. Write **Your Name** and **Roll No** on your Paper/Sheet's first page.
3. **Do not copy from any source otherwise you will be penalized with negative marks.**
4. Complete your lab **within given Time Slot**.
5. Paste all your codes along with screenshots in a word file and renamed with your roll number.
6. Keep all your source files in your computer for verification. Do not overwrite a single source file for all programs.

Problems:

1. Create a program that generates random passwords with various characteristics, such as a specified length and a mix of uppercase letters, lowercase letters, numbers, and special characters. To make the code modular and reusable, you decide to design a PasswordGenerator class.

Solution Using OOP:

Class Name: PasswordGenerator

Attributes:

- **length** (integer): The desired length of the password.
- **character_sets** (list of strings): A list of character sets to include in the password, such as uppercase letters, lowercase letters, digits, and special characters.

Methods:

1. **__init__(self, length=12)**: The constructor initializes the **length** attribute with the default value of 12. It also initializes the **character_sets** attribute to contain all character sets (uppercase letters, lowercase letters, digits, and special characters).
2. **set_length(self, length)**: Allows setting a new length for the password.



3. **set_character_sets(self, character_sets)**: Allows specifying which character sets to include in the password. This method takes a list of strings, where each string represents a character set (e.g., "upper", "lower", "digits", "special").
 4. **generate_password(self)**: Generates a random password based on the specified length and character sets. It returns the generated password as a string.
-
2. Create a program that analyzes a text file to count the occurrences of each word and then displays the top N most frequent words in the text. To make the code modular and reusable, you decide to design a TextAnalyzer class.

Solution Using OOP:

Class Name: TextAnalyzer

Attributes:

- **file_path** (string): The path to the text file to be analyzed.
- **word_count** (dictionary): A dictionary that stores word frequencies, with words as keys and their counts as values.

Methods:

1. **__init__(self, file_path)**: The constructor initializes the **file_path** attribute with the provided path to the text file. It also initializes an empty **word_count** dictionary.
2. **read_file(self)**: Reads the text file specified by **file_path** and processes its content. This method tokenizes the text into words, removes punctuation, and updates the **word_count** dictionary with word frequencies.
3. **count_words(self)**: Counts the occurrences of each word in the text and updates the **word_count** dictionary.
4. **get_top_words(self, n)**: Retrieves the top N most frequent words from the **word_count** dictionary and returns them as a list of tuples (word, count).
5. **display_top_words(self, n)**: Displays the top N most frequent words in a user-friendly format, such as word: count.

3. Create a Hangman game in which the computer selects a random word, and the user tries to guess it letter by letter. To make the code modular and organized, you decide to design a HangmanGame class.

Solution Using OOP:

Class Name: HangmanGame

Attributes:

- **word_to_guess** (string): The random word selected by the computer.
- **guessed_letters** (set): A set to store the letters guessed by the user.
- **max_attempts** (integer): The maximum number of attempts (guesses) allowed.
- **attempts_left** (integer): The number of attempts remaining.
- **is_won** (boolean): Indicates whether the game is won.
- **is_lost** (boolean): Indicates whether the game is lost.

Methods:

1. **__init__(self, word_list, max_attempts=6)**: The constructor initializes the game by selecting a random word from a word list (provided as an argument) and sets the maximum number of attempts. It also initializes other attributes such as **guessed_letters**, **attempts_left**, **is_won**, and **is_lost**.
2. **display_word(self)**: Displays the word with letters replaced by underscores for hidden letters and reveals correctly guessed letters.
3. **make_guess(self, letter)**: Allows the user to make a letter guess. It updates **guessed_letters**, checks if the guessed letter is correct, and updates **attempts_left**. It also checks if the game is won or lost.
4. **is_game_over(self)**: Checks if the game is over (either won or lost).
5. **play(self)**: The main game loop where the user plays the game. It prompts the user for guesses until the game is over.

4. Create a program that allows users to perform various operations on lists, including adding elements, removing elements, finding the maximum value, and finding the minimum value. To make the code modular and organized, you decide to design a **ListOperations** class.

Solution Using OOP:

Class Name: ListOperations

Attributes:

- **my_list** (list): The list on which operations will be performed.

Methods:

1. **__init__(self, initial_list=None)**: The constructor initializes the **my_list** attribute. If an initial list is provided as an argument, it sets **my_list** to that list; otherwise, it initializes an empty list.
2. **add_element(self, element)**: Allows users to add an element to the end of the list.
3. **remove_element(self, element)**: Allows users to remove the first occurrence of a specified element from the list.
4. **find_maximum(self)**: Finds and returns the maximum value in the list.
5. **find_minimum(self)**: Finds and returns the minimum value in the list.

6. Create a number guessing game in which the computer selects a random number, and the user tries to guess it. The user should receive hints like "too high" or "too low" after each guess. To make the code modular and organized, you decide to design a **NumberGuessingGame** class.

Solution Using OOP:

Class Name: NumberGuessingGame

Attributes:

- **random_number** (integer): The randomly selected number by the computer.
- **max_attempts** (integer): The maximum number of attempts allowed.
- **attempts_left** (integer): The number of attempts remaining.
- **is_won** (boolean): Indicates whether the game is won.

Methods:

1. **__init__(self, max_number, max_attempts)**: The constructor initializes the game by selecting a random number between 1 and **max_number** and sets the maximum number of attempts. It also initializes other attributes such as **attempts_left** and **is_won**.
2. **guess_number(self, user_guess)**: Allows the user to make a guess. It updates **attempts_left**, checks if the guess is correct, and provides hints like "too high" or "too low." It also checks if the game is won or lost.
3. **is_game_over(self)**: Checks if the game is over (either won or lost).
4. **play(self)**: The main game loop where the user plays the game. It prompts the user for guesses until the game is over.

You need to do your exercise within the given time.